

```

1  -- =====
2  -- SECTION D : Joins Queries
3  -- Database   : Celebal_Week2
4  -- Table      : dbo.Superstore (single flat table - self-join
5  --              and derived-table joins used to simulate
6  --              multi-table relational analysis)
7  -- Tool       : SQL Server Management Studio (SSMS)
8  -- Author     : ARATI THORAT | Celebal Internship - Week 2
9  -- =====
10
11 USE Celebal_Week2;
12 GO
13
14 -- NOTE -----
15 -- Since the Superstore CSV was imported as a single flat table,
16 -- we use SELF-JOINS and SUBQUERY-based derived tables to
17 -- demonstrate INNER JOIN and LEFT JOIN logic meaningfully.
18 -- This mirrors real-world scenarios where one large table
19 -- is compared against itself or against aggregated views.
20 -- -----
21
22
23 -- -----
24 -- D1. INNER JOIN Simulation
25 --     PURPOSE : Join customer orders with their regional average
26 --               to compare individual orders vs regional baseline
27 -- -----
28 SELECT
29     s.Order_ID,
30     s.Customer_Name,
31     s.Region,
32     s.Category,
33     s.Product_Name,
34     ROUND(s.Sales, 2) AS Order_Sales,
35     ROUND(s.Profit, 2) AS Order_Profit,
36     ROUND(ra.Region_Avg, 2) AS Regional_Avg_Sales,
37     CASE
38         WHEN s.Sales > ra.Region_Avg
39         THEN 'Above Average'
40         ELSE 'Below Average'
41     END AS Sales_vs_Region
42 FROM dbo.Superstore AS s
43 INNER JOIN (
44     -- Derived table: average sales per region
45     SELECT Region, AVG(Sales) AS Region_Avg
46     FROM   dbo.Superstore
47     GROUP BY Region
48 ) AS ra ON s.Region = ra.Region
49 ORDER BY s.Region, s.Sales DESC;

```

```
50
51  /*
52     OBSERVATION:
53     West region has the highest regional average sale.
54     Most high-value Technology orders are 'Above Average' within
55     their region - confirming Tech is the growth driver everywhere.
56  */
57
58
59  -- -----
60  -- D2. INNER JOIN - Order Detail with Category Average
61  --     PURPOSE : Find orders that exceed the category average sale
62  -- -----
63  SELECT
64      s.Order_ID,
65      s.Order_Date,
66      s.Customer_Name,
67      s.Category,
68      s.Product_Name,
69      ROUND(s.Sales, 2) AS Order_Sales,
70      ROUND(ca.Cat_Avg, 2) AS Category_Avg_Sales,
71      ROUND(s.Profit, 2) AS Profit
72  FROM  dbo.Superstore AS s
73  INNER JOIN (
74      SELECT Category, AVG(Sales) AS Cat_Avg
75      FROM    dbo.Superstore
76      GROUP BY Category
77  ) AS ca ON s.Category = ca.Category
78  WHERE s.Sales > ca.Cat_Avg
79  ORDER BY s.Category, s.Sales DESC;
80
81  /*
82     OBSERVATION:
83     Technology orders that exceed the category average are mostly
84     Copiers and Phones - the two highest-ticket sub-categories.
85     Furniture orders exceeding the category average are Bookcases
86     and Tables, but many still have negative profit.
87  */
88
89
90  -- -----
91  -- D3. INNER JOIN - Customer's Orders vs Their Own Average
92  --     PURPOSE : Find customers with unusually large orders
93  -- -----
94  SELECT
95      s.Order_ID,
96      s.Order_Date,
97      s.Customer_Name,
98      s.Segment,
```

```

99     s.Region,
100     ROUND(s.Sales,          2) AS Order_Sales,
101     ROUND(cust.Cust_Avg,    2) AS Customer_Avg_Sales
102 FROM dbo.Superstore AS s
103 INNER JOIN (
104     SELECT Customer_ID, AVG(Sales) AS Cust_Avg
105     FROM   dbo.Superstore
106     GROUP BY Customer_ID
107 ) AS cust ON s.Customer_ID = cust.Customer_ID
108 WHERE s.Sales > cust.Cust_Avg * 2  -- orders 2x the customer's own
      average
109 ORDER BY s.Sales DESC;
110
111 /*
112  OBSERVATION:
113  These are "spike" orders – unusual one-time large purchases.
114  Useful for detecting bulk B2B orders or anomalous transactions.
115 */
116
117
118 -- -----
119 -- D4. INNER JOIN – Region × Sub-Category Performance Matrix
120 --   PURPOSE : Cross-dimensional analysis of region and product
121 -- -----
122 SELECT
123     s.Region,
124     s.Sub_Category,
125     rs.Total_Region_Sales,
126     ROUND(SUM(s.Sales), 2) AS SubCat_Sales_In_Region,
127     ROUND(SUM(s.Profit), 2) AS SubCat_Profit_In_Region,
128     ROUND(SUM(s.Sales) * 100.0
129           / NULLIF(rs.Total_Region_Sales, 0), 2) AS Pct_Of_Region_Sales
130 FROM dbo.Superstore AS s
131 INNER JOIN (
132     SELECT Region, SUM(Sales) AS Total_Region_Sales
133     FROM   dbo.Superstore
134     GROUP BY Region
135 ) AS rs ON s.Region = rs.Region
136 GROUP BY s.Region, s.Sub_Category, rs.Total_Region_Sales
137 ORDER BY s.Region, SubCat_Sales_In_Region DESC;
138
139 /*
140  OBSERVATION:
141  Phones and Chairs are top sub-categories in every region.
142  Tables consistently underperform profit-wise across ALL regions.
143  The Central region has the lowest % contribution from
144  high-margin Technology sub-categories.
145 */
146

```

```
147
148 -- -----
149 -- D5. LEFT JOIN Simulation - Customers with Below-Average Profit
150 --     PURPOSE : Find customers contributing less than average
151 -- -----
152
153 -- Step 1: Identify all customers and their total profit
154 -- Step 2: LEFT JOIN to overall average to flag underperformers
155
156 SELECT
157     cust.Customer_ID,
158     cust.Customer_Name,
159     cust.Segment,
160     cust.Region,
161     ROUND(cust.Total_Sales, 2) AS Total_Sales,
162     ROUND(cust.Total_Profit, 2) AS Total_Profit,
163     ROUND(avg_profit.Avg_Profit, 2) AS Company_Avg_Profit,
164     CASE
165         WHEN cust.Total_Profit < 0
166         THEN 'Loss-Generating Customer'
167         WHEN cust.Total_Profit < avg_profit.Avg_Profit
168         THEN 'Below Average Profit'
169         ELSE 'Above Average Profit'
170     END AS Customer_Profit_Status
171 FROM (
172     SELECT Customer_ID, Customer_Name, Segment, Region,
173            SUM(Sales) AS Total_Sales,
174            SUM(Profit) AS Total_Profit
175     FROM   dbo.Superstore
176     GROUP BY Customer_ID, Customer_Name, Segment, Region
177 ) AS cust
178 LEFT JOIN (
179     SELECT AVG(cust_inner.Total_Profit) AS Avg_Profit
180     FROM (
181         SELECT Customer_ID, SUM(Profit) AS Total_Profit
182         FROM   dbo.Superstore
183         GROUP BY Customer_ID
184     ) AS cust_inner
185 ) AS avg_profit ON 1 = 1
186 ORDER BY cust.Total_Profit ASC;
187
188 /*
189     OBSERVATION:
190     Some customers have placed many orders yet generate NET LOSSES.
191     These are often customers who consistently request high discounts.
192     LEFT JOIN ensures ALL customers appear even if they don't match
193     a filter - important for complete auditing.
194 */
195
```

```
196
197 -- -----
198 -- D6. INNER JOIN – Year-over-Year Order Comparison (Self-Join)
199 --     PURPOSE : Compare a customer's orders across two years
200 -- -----
201 SELECT
202     y1.Customer_Name,
203     y1.Segment,
204     ROUND(y1.Sales_2016, 2) AS Sales_2016,
205     ROUND(y2.Sales_2017, 2) AS Sales_2017,
206     ROUND(y2.Sales_2017 - y1.Sales_2016, 2) AS YoY_Growth
207 FROM (
208     SELECT Customer_ID, Customer_Name, Segment, SUM(Sales) AS Sales_2016
209     FROM   dbo.Superstore
210     WHERE  YEAR(Order_Date) = 2016
211     GROUP BY Customer_ID, Customer_Name, Segment
212 ) AS y1
213 INNER JOIN (
214     SELECT Customer_ID, SUM(Sales) AS Sales_2017
215     FROM   dbo.Superstore
216     WHERE  YEAR(Order_Date) = 2017
217     GROUP BY Customer_ID
218 ) AS y2 ON y1.Customer_ID = y2.Customer_ID
219 ORDER BY YoY_Growth DESC;
220
221 /*
222     OBSERVATION:
223     INNER JOIN here returns only customers who placed orders in
224     BOTH 2016 AND 2017 – i.e., returning customers.
225     Customers showing positive YoY growth are loyal, growing accounts.
226     Customers with negative growth might be churning.
227 */
228
229
230 -- -----
231 -- D7. LEFT JOIN – Customers who ordered in 2016 but NOT 2017
232 --     PURPOSE : Identify churned customers
233 -- -----
234 SELECT
235     prev.Customer_ID,
236     prev.Customer_Name,
237     prev.Segment,
238     ROUND(prev.Sales_2016, 2) AS Sales_2016,
239     curr.Sales_2017
240 FROM (
241     SELECT Customer_ID, Customer_Name, Segment, SUM(Sales) AS Sales_2016
242     FROM   dbo.Superstore
243     WHERE  YEAR(Order_Date) = 2016
244     GROUP BY Customer_ID, Customer_Name, Segment
```

```

245 ) AS prev
246 LEFT JOIN (
247     SELECT Customer_ID, SUM(Sales) AS Sales_2017
248     FROM     dbo.Superstore
249     WHERE    YEAR(Order_Date) = 2017
250     GROUP BY Customer_ID
251 ) AS curr ON prev.Customer_ID = curr.Customer_ID
252 WHERE curr.Customer_ID IS NULL    -- ordered in 2016 but NOT in 2017
253 ORDER BY prev.Sales_2016 DESC;
254
255 /*
256     OBSERVATION:
257     LEFT JOIN is key here – it keeps ALL 2016 customers and
258     identifies those with NO matching 2017 orders (NULL values).
259     These are potentially churned customers and prime targets
260     for re-engagement campaigns.
261     Some high-value 2016 customers are not seen in 2017 –
262     a significant revenue risk.
263 */
264
265
266 -- -----
267 -- D8. INNER JOIN – Segment × Region cross analysis
268 --     PURPOSE : Full cross-dimensional business intelligence view
269 -- -----
270 SELECT
271     s.Segment,
272     s.Region,
273     ROUND(SUM(s.Sales), 2)                AS Total_Sales,
274     ROUND(SUM(s.Profit), 2)               AS Total_Profit,
275     COUNT(DISTINCT s.Order_ID)           AS Total_Orders,
276     ROUND(SUM(s.Sales) / COUNT(DISTINCT s.Order_ID), 2) AS
277         Avg_Order_Value,
278     ROUND(SUM(s.Profit) * 100.0
279         / NULLIF(SUM(s.Sales), 0), 2)    AS Profit_Margin_Pct
279 FROM dbo.Superstore AS s
280 INNER JOIN (
281     SELECT DISTINCT Segment, Region
282     FROM     dbo.Superstore
283 ) AS seg_reg ON s.Segment = seg_reg.Segment
284              AND s.Region = seg_reg.Region
285 GROUP BY s.Segment, s.Region
286 ORDER BY s.Segment, Total_Sales DESC;
287
288 /*
289     OBSERVATION:
290     Best combination  : Corporate + West → highest profit margin
291     Worst combination : Consumer + Central → negative margins possible
292     Home Office + East shows surprisingly strong average order values.

```

293 */